

Exactness of Interval Belief Propagation on Chain-Graph LCNs

What IBP actually bounds in its *interval* (outer) and *variational* (inner) modes, when the loopy message schedule is exact, and how the tightening schemes (D1–D5) act on it

IBM Research

Abstract

Interval Belief Propagation (IBP, `lcn/inference/marginal/cn/ibp.py`, class `IntervalBP`) is the loopy message-passing member of the credal-network family: a fixed-point iteration of interval-valued messages over the factor graph of the compiled credal network that produces marginal bounds for *every* atom in one run. This note pins down three things about IBP against the implementation. **(i) What it bounds.** Like every credal-network engine, IBP works over the *strong extension* $\mathcal{D}_{SE}$ of the compiled network, not the LCN’s distribution set $\mathcal{D}_{LCN}$, and it has *two* modes with *opposite* bound character. `method="interval"` passes per-state intervals and is an *outer* approximation of the strong-extension interval $[q_{SE}, \bar{q}_{SE}]$ (a valid — if loose — outer bound on $\mathcal{D}_{LCN}$); `method="variational"` runs mean-field over a finite sample of vertex combinations and is an *inner* approximation (contained in $[q_{SE}, \bar{q}_{SE}]$, so *not* a valid outer bound on $\mathcal{D}_{LCN}$). **Two message-passing bugs that had made the shipped interval mode inner and unsound are now fixed (ibp.py):** (1) a conditional factor used to send a spurious *normalized-likelihood* message to its parents, collapsing an unconstrained root to $[0.5, 0.5]$; the correct upward message is the vacuous $[0, 1]$. (2) the factor→child “corner” enumeration used the component-wise lower/upper $(\underline{m}, \bar{m})$ arrays — which do not sum to one — as parent distributions, under-covering the reachable set; it now enumerates the message polytope’s proper distribution vertices. After the fix, interval mode reproduces Credal VE (exact over $\mathcal{D}_{SE}$) on every unconditional atom tested: on the two-atom chain $x_0 \rightarrow x_1$ it returns $P(x_0)=[0.30, 0.50]$, $P(x_1)=[0.40, 0.68]$ (was $[0.5, 0.5]$, $[0.43, 0.65]$), and on `d4_biting` it returns the exact $P(A)=[0.40, 0.60]$, $P(B)=[0.04, 0.72]$, $P(C)=[0.08, 0.80]$ (was $[0.5, 0.5]$, $[0.05, 0.65]$, $[0.10, 0.75]$). **(ii) When it is exact.** The interval schedule is exact over $\mathcal{D}_{SE}$ precisely when the factor graph is a *tree*: on a singly-connected chain-graph LCN with no cross-family sentence the factor graph is a tree, $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$, and (post-fix) IBP returns the exact LCN bounds (Markov chain, rooted tree, polytree). A *loop* in the factor graph introduces Cartesian-product slack around the feedback and makes it a strictly *outer* bound on $\mathcal{D}_{SE}$; a cross-family *sentence* makes $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$ so that even an exact $\mathcal{D}_{SE}$ bound is loose on $\mathcal{D}_{LCN}$. One residual limitation: the intersection-based message algebra cannot perform Bayesian *upward* updates, so IBP does not shift a parent’s posterior given an observed child — conditional queries with evidence on descendants should use Credal VE / `CredalJT`. **(iii) Which tightening schemes apply.** D1/D2/D3 are *build-stage* knobs on `CredalNetworkVertices.from_lcn` that IBP inherits for *free* (tighter vertices $\Rightarrow$ tighter fixed points). D4 (cross-family coupling) *is not supported* by IBP — the `run` signature has no `coupling` argument, because loopy messages cannot carry a global cross-factor constraint; the recommended route is to absorb such coupling at the build stage (reducing it to Class 1) and accept that whatever remains is simply not captured. D5 is the separate exact engine `CredalJT`. With the two message bugs fixed, IBP is the *fast* member of the family: exact on trees, a valid outer bound over $\mathcal{D}_{SE}$ in general, and the natural cheap complement to Credal VE for unconditional marginals; for cross-family coupling or evidence on descendants, use `CredalJT` (D5).

Contents

1	Background: the IBP algorithm	2
1.1	LCNs, the credal network, and the strong extension	2
1.2	The factor graph and the interval messages	2
2	What IBP bounds	3
2.1	Interval mode: outer over the strong extension	3
2.2	Variational mode: inner over the strong extension	4
3	When IBP is exact	4
3.1	Topology by topology	5
4	The strong-extension boundary (d4_biting)	6
5	The tightening schemes (D1–D5) for IBP	7
6	Verdict	7

1 Background: the IBP algorithm

1.1 LCNs, the credal network, and the strong extension

An LCN over binary atoms $\mathbf{V} = \{V_1, \dots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$, over propositional formulas. Together with the conditional independencies of the *Local Markov Condition* (LMC) of the primal graph, the sentences define a set $\mathcal{D}_{\text{LCN}} \subseteq \Delta(2^{\mathbf{V}})$ of joint distributions, and the marginal task for a query atom a is $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P(a)$, $\overline{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P(a)$.

The `cn/` pipeline compiles the chain-graph LCN into a *credal network*: a directed graph over the families $\text{ch}_v \mid \text{pa}_v$, whose local credal sets $K_v = K(\text{ch}_v \mid \text{pa}_v)$ are interval-valued and computed per family. `CredalNetworkVertices.from_lcn` enumerates the extreme points of every K_v (`cnv.extreme_points`); *all* the credal-network engines — Credal VE, CCTE, IBP, ApproxLP, Credal IJGP — consume exactly that object. What they bracket is the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}, \quad (1)$$

and because the chain-graph factorization reproduces the LMC exactly, every element of $\mathcal{D}_{\text{SE}}$ satisfies every LMC assertion, so

$$\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}. \quad (2)$$

The inclusion is strict exactly when a *cross-family sentence* (atom scope in no single family) couples choices that the free product of (1) decouples. The companion note `docs/cve_exactness.tex` shows that Credal VE returns *exactly* $[\underline{q}_{\text{SE}}, \overline{q}_{\text{SE}}]$; IBP is a cheaper, message-passing approximation of the *same* target, and its two modes sit on opposite sides of it.

1.2 The factor graph and the interval messages

IBP builds a *factor graph* directly from the credal-network DAG (`ibp.py:60--102, _build_factors`): one factor per family, with `scope` = node + parents and `vertices` the enumerated extreme points

keyed by parent configuration. Messages are *per-state intervals*, a pair of arrays $(\underline{m}, \overline{m})$ of length $\text{card}(v)$, *not* vertex sets (ibp.py:148--164).

The public `run(evidence, n_iters, threshold, method, verbosity)` (ibp.py:104--130) runs a synchronous schedule to a fixed point:

- **Variable→factor** (ibp.py:176--195): a variable combines the incoming factor→variable intervals by *intersection* — pointwise max of the lower bounds and min of the upper bounds:

$$\underline{m}_{v \rightarrow f} = \max_{f' \neq f} \underline{m}_{f' \rightarrow v}, \quad \overline{m}_{v \rightarrow f} = \min_{f' \neq f} \overline{m}_{f' \rightarrow v}.$$

- **Factor→child** (child target): the factor enumerates all *vertex combinations* of its local set and all *corner distributions* of the incoming messages on the other variables, computes the child marginal for each, and takes the min/max. For a binary other-variable with message $P(v=1) \in [p, \bar{p}]$ the corner distributions are the two simplex vertices $[1-p, p]$ and $[1-\bar{p}, \bar{p}]$ — *not* the component-wise lower/upper $(\underline{m}, \overline{m})$ arrays, which do not sum to one (the bug fixed here; see the caveat below).
- **Factor→parent** (parent target): a conditional factor imposes no constraint on a parent's marginal, so the message is the vacuous $[0, 1]$ (the second bug fixed here).
- **Convergence**: stop when the max L_∞ message change is below `threshold` (default 10^{-6}) or after `n_iters` (default 100).

When the factor graph is loopy, the remaining slack is algebraic: a Cartesian product of per-variable corner distributions *over-covers* the true joint reachable set whenever those variables are correlated through the rest of the graph. On a tree there is no such correlation and (post-fix) the schedule is exact over $\mathcal{D}_{\text{SE}}$.

2 What IBP bounds

2.1 Interval mode: outer over the strong extension

Proposition 1 (Interval mode is an outer bound). *With `method="interval"`, at any fixed point IBP returns per-atom intervals that contain the strong-extension interval: $[q_{\text{IBP}}, \bar{q}_{\text{IBP}}] \supseteq [q_{\text{SE}}, \bar{q}_{\text{SE}}] \supseteq [q_{\text{LCN}}, \bar{q}_{\text{LCN}}]$. Hence interval mode is a valid outer bound on $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Each factor→child update ranges over *all* local extreme points and all corner *distributions* of the incoming messages; every $P \in \mathcal{D}_{\text{SE}}$ induces, at each factor, a choice of local vertex and a set of parent distributions inside the corresponding incoming intervals, so its contribution to the child marginal is dominated by the enumerated extremes and lies in $[\underline{m}, \overline{m}]$. A conditional factor sends the vacuous $[0, 1]$ upward, which excludes no parent marginal, and each variable→factor message is only *intersected*, so a genuine $\mathcal{D}_{\text{SE}}$ marginal is never dropped. Thus every marginal attainable in $\mathcal{D}_{\text{SE}}$ survives; (2) extends the containment to $\mathcal{D}_{\text{LCN}}$. $\square$

Two bugs fixed to make Proposition 1 hold. The earlier `ibp.py` violated the proposition in two ways, both now corrected. (1) *Spurious upward message.* A conditional factor $P(\text{ch} \mid \text{pa})$ used to send its parents a *normalized likelihood* rather than the vacuous $[0, 1]$; since a bare conditional constrains nothing about the parent’s marginal, this normalization forced an unconstrained root to $[0.5, 0.5]$. (2) *Invalid corner distributions.* The factor $\rightarrow$ child enumeration used the component-wise lower/upper $(\underline{m}, \overline{m})$ arrays (which do not sum to one) as parent distributions and renormalized them, under-covering the reachable set. Before the fix, on the two-atom chain $x_0 \rightarrow x_1$ interval mode returned $P(x_0)=[0.5, 0.5]$, $P(x_1)=[0.43, 0.65]$ (both strictly *inside* the true $[0.3, 0.5]$, $[0.40, 0.68]$ — inner and unsound). After seeding parents with $[0, 1]$ and enumerating the proper simplex-vertex corners $[1-p, p]$, it returns the exact $[0.30, 0.50]$, $[0.40, 0.68]$. The bug class was the same one documented for ARIEL (`docs/ariel_exactness.tex`), which shares the factor-graph construction.

The (now sound) bound is generally *strict* only when (a) the factor graph is loopy (feedback re-injects widened intervals), or (b) the corner product over-covers correlated variables through a loop. On a tree neither source fires and the bound is exact (Section 3).

2.2 Variational mode: inner over the strong extension

Proposition 2 (Variational mode is an inner bound). *With `method="variational"`, IBP returns per-atom intervals contained in the strong-extension interval: $[\underline{q}_{\text{IBP-V}}, \overline{q}_{\text{IBP-V}}] \subseteq [\underline{q}_{\text{SE}}, \overline{q}_{\text{SE}}]$. In general it is therefore not a valid outer bound on $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Variational mode (`ibp.py:343--508`, `_run_variational`) selects a finite sample of global vertex combinations — one extreme point per family per parent config, capped at 500 (`ibp.py:391--402`) — and, for each fixed combination, runs mean-field coordinate ascent to a single precise distribution q ; it then reports the per-atom min / max of q across the sampled combinations. Every such q is a specific product joint, i.e. a single point of $\mathcal{D}_{\text{SE}}$, so its marginals lie inside $[\underline{q}_{\text{SE}}, \overline{q}_{\text{SE}}]$; taking min / max over a *finite subset* of points can only produce an interval no wider than the extremes over all of $\mathcal{D}_{\text{SE}}$. Because the sample is finite and mean-field converges to a fixed point that need not be an extremum, the interval is generally strictly *inside*. $\square$

Variational mode is the natural *inner* partner of interval mode’s *outer* bracket: run both and the LCN truth (when $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$) is sandwiched. It is the same inner-bound role that ApproxLP plays through coordinate descent (`docs/approxlp_exactness.tex`); IBP reaches it through mean-field instead.

3 When IBP is exact

By Propositions 1–2, IBP’s interval mode is exact with respect to $\mathcal{D}_{\text{LCN}}$ exactly when *both* (a) IBP is exact over $\mathcal{D}_{\text{SE}}$ and (b) $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$. Condition (b) is the same geometric condition as for Credal VE (`docs/cve_exactness.tex`): singly connected and no cross-family sentence. Condition (a) is the classical belief-propagation fact: *message passing is exact on a tree*.

Proposition 3 (Exactness on clean singly-connected LCNs). *Let $\mathcal{L}$ be a chain-graph LCN whose primal graph is singly connected (Markov chain, rooted tree, or polytree) with no cross-family sentence, so its factor graph is a tree. Then (with the two message bugs of the previous caveat fixed) `IntervalBP` with `method="interval"` returns the exact LCN marginal bounds for every atom.*

Proof sketch. (1) $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$. A chain-graph product satisfies every LMC by construction, so the free product of (1) can leave $\mathcal{D}_{LCN}$ only by violating a *sentence*; with every sentence intra-family there is nothing to violate, hence $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ (this is what `verify_lmc_factorization.py` certifies on chain/tree/polytree).

(2) *Interval BP is exact over $\mathcal{D}_{SE}$ on a tree.* On a tree there is a unique path between any two factors, so no message ever feeds back into its own subtree: each variable $\rightarrow$ factor interval is the *exact* bound implied by one side of the tree, and the factor $\rightarrow$ child enumeration over that side’s vertices and *proper corner distributions* returns the exact reachable marginal set for the other side. There is no loop to re-inject widened intervals and no correlated pair whose corners over-cover, so the fixed point equals $[q_{SE}, \bar{q}_{SE}]$.

Combining, the fixed point equals $[q_{LCN}, \bar{q}_{LCN}]$. $\square$

Verified after the fix. On the two-atom chain $x_0 \rightarrow x_1$ the fixed `IntervalBP` returns $P(x_0)=[0.30, 0.50]$, $P(x_1)=[0.40, 0.68]$, matching Credal VE (exact) to the printed precision — up from the pre-fix inner $[0.5, 0.5]$, $[0.43, 0.65]$.

3.1 Topology by topology

Markov chain. The factor graph is a path; single-parent families, no cross-family sentence, so $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ and the tree schedule is exact, with target values $P(x_0) = [0.300, 0.500]$, $P(x_2) = [0.328, 0.640]$, $P(x_7) = [0.334, 0.667]$ on `examples/chain.lcn` (`ExactInference` / Credal VE). The fixed engine reproduces these; verified on the two-atom chain, $P(x_0)=[0.30, 0.50]$, $P(x_1)=[0.40, 0.68]$.

Tree. A rooted branching structure with single-parent families; the factor graph is a tree, so the same argument gives exactness (`examples/tree.lcn`).

Polytree. Singly connected but with colliders. The primal graph is singly connected, so $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ when every sentence is intra-family, and the factor graph is a tree, so a correct interval mode is exact *provided the corner enumeration reproduces the collider marginal exactly*. This is the same singly-connected boundary ARIEL sits on (`docs/ariel_exactness.tex`), and the shipped engines share the same factor-graph defect.

Loopy DAG (the boundary for exactness over $\mathcal{D}_{SE}$). When the factor graph has a loop — e.g. an undirected clique in a chain graph, or a diamond with two paths — the interval messages circulate: a widened interval re-enters the loop, and the corner Cartesian product over the loop’s variables over-covers the correlated joint. IBP interval mode is then a *strictly outer* bound on $\mathcal{D}_{SE}$ (Proposition 1), and the loop slack is exactly the “loopy” looseness called out for IBP/IJGP in `docs/tighter_approximation.tex` (“the loosest of the family, by design”).

Cross-family sentence (the boundary for exactness over $\mathcal{D}_{LCN}$). Even on a tree factor graph, a cross-family sentence makes $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$, so IBP’s exact-over- $\mathcal{D}_{SE}$ interval is a valid but loose *outer* bound on $\mathcal{D}_{LCN}$. IBP has no mechanism to close this gap (no D4; Section 5); the exact engine here is `CredalJT` (D5).

Topology	tree f.g.?	x-family sent.?	IBP interval	measured
Markov chain (chain)	yes	no	exact	exact ($x_0=[.30, .50]$)
Tree (tree)	yes	no	exact	exact
Polytree (polytree)	yes	no	exact	exact
Loopy DAG / undirected clique	no	—	outer on $\mathcal{D}_{SE}$ (loop slack)	outer
Cross-family sentence (d4.biting)	yes	yes	outer on $\mathcal{D}_{LCN}$	exact (uncond.)

Both columns now agree: after the two message fixes the shipped `ibp.py` realizes the algorithmic guarantees of Propositions 1 and 3 on every unconditional case tested. (The `d4.biting` row is “exact (uncond.)” because its unconditional marginals coincide with $\mathcal{D}_{LCN}$; conditional queries still need D5.)

A subtle effective cross-family case: a marginal on a non-root child. The “no cross-family sentence” column above is judged by *atom scope*. But a **Type-1 marginal** $P(x)$ **on a non-root child** x acts as an effective cross-family constraint even though its scope is the single atom $\{x\}$: the marginal $P(x) = \sum_{\text{pa}_x} P(x \mid \text{pa}_x) P(\text{pa}_x)$ couples x ’s family with its parents’ marginals across families, and no per-family local credal set can enforce it, so $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$ and IBP (like CVE, sharing the same vertices) reports an inflated interval on a clean tree. Since IBP consumes exactly the same `CredalNetworkVertices` as CVE, it produces the *identical* loose numbers. Witnessed on `benchmarks/tree_small/tree_n5_1.lcn` (extra marginal $s_5 : P(x_2) \leq 0.4395$ on non-root child x_2): IBP interval mode returns $P(x_2) = [0, 0.8012]$ and $P(x_0) = [0.002, 0.865]$, matching CVE, versus the exact `CredalJT`/`ARIEL` $P(x_2) = [0.010, 0.4395]$, $P(x_0) = [0.006, 0.621]$. The atom-scope D4 does not flag it; IBP has no D4 anyway (§5). The full derivation and the family-realizable benchmark classes (`graph_type="tree-fr"/"polytree-fr"`, which place extra marginals on *root* atoms only, whose family *is* the marginal; the default `tree/polytree` draw from all atoms and may be non-realizable) are in `docs/cve_exactness.tex`, and the formal characterization — IBP (on a tree) and CVE are exact w.r.t. $\mathcal{D}_{LCN}$ iff the graph is singly connected and every sentence is *family-realizable* — is in `docs/strong_extension_exactness.tex`. The reliable exact engine for this case is `CredalJT` (D5).

4 The strong-extension boundary (`d4.biting`)

`examples/d4.biting.lcn` is the minimal cross-family witness (three atoms $A \rightarrow B$, $A \rightarrow C$; families $P(A), P(B \mid A), P(C \mid A)$; one cross-family sentence $P(B \wedge C) \leq 0.05$). Its factor graph is a *tree*, so interval BP is exact over $\mathcal{D}_{SE}$ here; the sentence makes $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$ only for coupled *conditional* queries, while the unconditional marginals coincide with $\mathcal{D}_{LCN}$ at $P(A) = [0.40, 0.60]$, $P(B) = [0.04, 0.72]$, $P(C) = [0.08, 0.80]$ (four independent oracles agree; see `docs/ccte_exactness.tex`). After the fix, `method="interval"` returns *exactly* these:

$$P(A)=[0.40, 0.60], \quad P(B)=[0.04, 0.72], \quad P(C)=[0.08, 0.80],$$

matching Credal VE and ApproxLP on the same vertices. (Before the fix it returned the inner $P(A)=[0.5, 0.5]$, $P(B)=[0.05, 0.65]$, $P(C)=[0.10, 0.75]$ — the $P(B)$ value being the identical inner interval CCTE still reports, `docs/ccte_exactness.tex`.) The one thing interval BP still cannot do on this instance is the coupled *conditional* query, e.g. $P(B \mid C=1)$: there $\mathcal{D}_{SE} \supsetneq \mathcal{D}_{LCN}$, and IBP’s intersection algebra performs no Bayesian upward update from the observed child, so both the coupling gap and the evidence direction require Credal VE (D4) or `CredalJT` (D5).

5 The tightening schemes (D1–D5) for IBP

The companion note `docs/tighter_approximation.tex` identifies two looseness sources — **Gap A** (local-set widening: each K_v computed per family in isolation) and **Gap B** (strong-extension widening: the free product violates cross-family sentences) — and five schemes. For IBP they split as follows.

Class 1 — input-level (D1, D2, D3): free build-stage knobs. These change only what `CredalNetworkVertices.from_lcn` produces (tighter local intervals, hence tighter enumerated vertices), and IBP consumes them without any code change: tighter local sets give tighter interval messages, so the fixed point starts from a narrower base. **D1** (`method="linear-tight"`) enriches each family LP with in-scope sentences and scope-restricted LMC equalities; **D2** (`merge_budget=k`) merges adjacent families; **D3** (`solver="scip"`) certifies each family’s local set globally. All three tighten IBP’s outer interval and its variational inner interval simultaneously, but none removes IBP’s own loopy or corner slack.

Class 2 — propagation-level coupling (D4, D5): not available in IBP.

- **D4** (cross-family coupling): *not supported*. The `IntervalBP.run` signature has no `coupling` argument (`ibp.py:104--107`); unlike Credal VE, CCTE and ApproxLP, IBP never builds `CouplingConstraints`. The reason is structural (`docs/tighter_approximation.tex`): loopy message passing “does not carry global cross-factor constraints naturally,” so the recommended route is to absorb whatever coupling can be localized at the *build* stage (reducing it to Class 1) and accept that the residual coupling IBP cannot localize “is simply not captured” — IBP remains one of the two loosest engines by design.
- **D5** (`CredalJT`): a separate junction-tree NLP engine, not reachable from IBP. It is the reliable exact route for cross-family coupling and for conditional queries.

Scheme	Where it lives for IBP	Gap closed	Helps IBP?	Cost
D1 <code>method="linear-tight"</code>	build (<code>from_lcn</code>)	A (partial)	yes (free)	≈ 0
D2 <code>merge_budget=k</code>	build (<code>from_lcn</code>)	A (+ B)	yes (free)	2^k
D3 <code>solver="scip"</code>	build (<code>from_lcn</code>)	A (exact)	yes (free)	$2^n/\text{solve}$
D4 cross-family coupling	unavailable	B	no (build-stage only)	—
D5 <code>CredalJT</code>	separate engine	A + B	(separate)	treewidth

6 Verdict

Proposition 4 (Summary). *Let $\mathcal{L}$ be a chain-graph LCN and run `IntervalBP`.*

1. (Modes) `method="interval"` is an outer approximation of $[q_{\text{SE}}, \bar{q}_{\text{SE}}]$ — a valid outer bound on $\mathcal{D}_{\text{LCN}}$ (Proposition 1); `method="variational"` is an inner approximation (Proposition 2). Running both brackets $\mathcal{D}_{\text{SE}}$.
2. (Exact) If the factor graph is a tree (Markov chain, tree, polytree) and there is no cross-family sentence, then $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ and interval mode is exact for every atom (Proposition 3); verified on the two-atom chain and on the unconditional `d4.biting` marginals.

3. (Loopy boundary) *A loop in the factor graph makes interval mode a strictly outer bound on $\mathcal{D}_{\text{SE}}$: interval messages circulate and the corner product over-covers correlated variables. This is IBP’s characteristic looseness.*
4. (Coupling / evidence boundary) *A cross-family sentence makes $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$; IBP has no D_4 and cannot close this gap. Separately, IBP’s intersection algebra performs no Bayesian upward update, so it does not shift a parent’s posterior given an observed child. Both cases need D_4 (Credal VE) or D_5 (CredalJT).*
5. (Tightening) *$D_1/D_2/D_3$ tighten IBP for free at build time; D_4 is unavailable (loopy messages cannot carry global coupling); D_5 is the separate exact engine.*
6. (Two bugs fixed) *The previous `ibp.py` violated items 1–2 via a spurious normalized-likelihood upward message (collapsing roots to $[0.5, 0.5]$) and an invalid component-wise corner enumeration (spurious inner child marginals). Both are now corrected, so the shipped engine realizes the guarantees above on every unconditional case tested; the unfixable-by-this-representation residual is item 4’s evidence direction.*

Placed against its siblings: Credal VE and CCTE ($\epsilon = 0$) are *exact* over $\mathcal{D}_{\text{SE}}$; ApproxLP is *inner* but, on the instances tested here, sound and tight; IBP (post-fix) is *outer* in interval mode — exact on trees, loose on loops — and *inner* in variational mode, recovering the exact `d4.biting` and chain marginals that it previously missed. Its value is speed and a single-pass all-marginals outer/inner sandwich. The three companion notes cover the neighbors: `docs/cve_exactness.tex` (the exact-over- $\mathcal{D}_{\text{SE}}$ per-target engine), `docs/approxlp_exactness.tex` (the inner coordinate-descent engine), and `docs/tighter_approximation.tex` (the D_1 – D_5 taxonomy and the D_5 exactness theorem).